

Changelog — helix_proxy-0.1.0-dev-0.0.2

Revision: 2 **Last modified:** 2026-07-01T11:23:22Z **Scope:** Every change on the release surface between the previous release tag helix_proxy-0.1.0-dev-0.0.1 (commit 3f79794, 2026-07-01 09:24:25 +0300) and the release candidate HEAD (2ed8f38, 2026-07-01 14:20:03 +0300) — 26 commits.

Revision 2 (2026-07-01T11:23:22Z): extended for the 12 commits that landed after the Rev-1 draft — the Let's Encrypt Phase-2/3 hermetic DNS-01 issuance lane (task #59, live-PROVEN) and the observability metrics lane (task #56, LIVE /metrics scrape PROVEN). New sections: the extended commit inventory rows 15–26, the LE + observability grouped entries, and a "Known-pending at this tag" note. **Authority:** Helix Constitution §11.4.151 (project-prefixed release tags), §11.4.135 (standing regression-guard suite), §11.4.44 (revision header), §11.4.65 (universal Markdown export), §11.4.6 (no-guessing).

Read-only planning/record artefact. It documents what landed; creating/pushing the tag is a separate, operator-authorized action gated by the §11.4.40 full-suite retest (see docs/releases/RELEASE_RUNBOOK.md).

Release-prefix note (§11.4.151)

The release prefix resolves deterministically, resolution order:

1. HELIX_RELEASE_PREFIX from .env — **unset** (no .env file exists in the checkout; .env is gitignored per §11.4.30, its template .env.example carries no HELIX_RELEASE_PREFIX).
2. Fallback = lowercased snake_case of the project-root directory basename = **helix_proxy**.

Resolved prefix: helix_proxy → this release is tagged helix_proxy-0.1.0-dev-0.0.2 (monotonic increment from -0.0.1), the SAME prefix carried by the previous tag, so git tag -l 'helix_proxy-*' enumerates the whole release surface. Optional hardening (runbook OB-5): declare

HELIX_RELEASE_PREFIX=helix_proxy in a gitignored .env + document it in the tracked .env.example (§11.4.77) so the prefix is authoritative rather than dir-name-derived.

Commit inventory (previous tag → HEAD, oldest first)

#	SHA	Type	Subject
1	9a21d94	feat	#54 — creds-drop-ready real-VPN-egress functional-proof harness + operator runbook
2	3308e5a	feat	#53 — optional separate plaintext /metrics listener (CONTROL_API_METRICS_ADDR)
3	2e834a7	fix	BUGFIX-0013 — CONST-033 scanner false-FAIL on governance-doc export siblings (F4)
4	f219b4d	fix	BUGFIX-0014 — anti-bluff proxy connectivity classifier (no false-FAIL on outage, no fail-open on crashed proxy)
5	8219669	fix	BUGFIX-0015 — require positive flood evidence before a ddos survival PASS
6	80bd833	fix	BUGFIX-0016 — real benchmark regression ratchet vs a committed baseline
7	fddd25c	docs	letsencrypt — design + phased plan for LE HTTPS with auto-renewal + rotation
8	9ffbc0a	fix	BUGFIX-0017 — de-bluff comprehensive-test.sh proxy canaries (F1)
9	39c4139	fix	BUGFIX-0018 — close assert_egress_ip host-undeterminable fail-open + register standing guards (F7 + F-1)
10	2812f48	feat	letsencrypt — Phase-1 offline cert-analyzer + self-validation guard + Status docs (task #59)
11	ae16f61	docs	continuation — §12.10 resume (anti-bluff sweep complete; LE Phase 1 landed)
12	86c1d8b	docs	releases — release runbook + prerequisite audit for helix_proxy-0.1.0-dev-0.0.2
13	f2e69e0	feat	challenges — live proxy Challenge bank (HTTP-forward + SOCKS5 + cache)
14	bf38c35	feat	helixqa — proxy test bank + runner with honest §11.4.3 harness-build SKIP
15	b2afa7d	feat	letsencrypt — Phase 2+3 hermetic DNS-01 issuance PROVEN end-to-end (task #59)
16	6349c0e	docs	

#	SHA	Type	Subject
			letsencrypt — hermetic DNS-01 + certmagic-#354 + renewal deep-research (§11.4.150)
17	4820da7	docs	status — feature-status ledger + this changelog (Rev 1) + LE hermetic security audit
18	022b78d	feat	challenges — LE Phase-3 hermetic issuance Challenge, live PASS (task #59)
19	d2042ac	test	letsencrypt — standing §11.4.135 Phase-3 issuance guard (RED+GREEN proven)
20	9b209e9	feat	observability — control-plane metrics overlay + RED_MODE scrape guard (task #56)
21	5d80c00	test	observability — hermetic mTLS test-cert generator for the #56 live-scrape
22	7c6b69f	docs	letsencrypt — Phase-5 renewal-mechanism correction (Caddy 2.8.4 admin-API JSON)
23	2f1e49d	docs	letsencrypt — Phase-5 live findings (ARI blocks renewal on Pebble 2.6.0)
24	401234e	feat	observability — isolated metrics trio, LIVE / metrics scrape PROVEN (task #56)
25	713d4dd	docs	letsencrypt — Pebble /set-renewal-info API + Phase-5 certmagic ARI-cache blocker
26	2ed8f38	docs	features — LE hermetic DNS-01 issuance = PASS in the feature ledger (task #59)

feat

- **9a21d94 — Real-VPN-egress functional-proof harness (#54).** A creds-drop-ready harness that, once WireGuard credentials are supplied, proves the §15 VPN-routing contract end-to-end (egress through the proxy == the expected tunnel exit AND != the host's real IP), with an operator runbook. Absent a live tunnel it SKIPS honestly (operator_attended, §11.4.52) — never a fabricated PASS.
- **3308e5a — Optional separate plaintext /metrics listener (#53).** New CONTROL_API_METRICS_ADDR config lets the control-API expose Prometheus /metrics on a separate plaintext listener, decoupled from the fail-closed mTLS control surface, for scrape topologies that cannot present a client cert.
- **2812f48 — Let's Encrypt Phase-1 offline cert-analyzer (task #59).** An offline certificate analyzer (tests/letsencrypt/cert_analyzer.sh) that is self-validating per §11.4.107(10): it ACCEPTS every golden-GOOD certificate property and REJECTS every golden-BAD one (expired / wrong-CA / wrong-

host / SAN-substring / not-due-for-renewal). Ships with a registered self-validation guard + per-feature Status docs. No network — Phase 2/3 (live issuance / auto-renewal / rotation) remain planned per the LE design.

- **f2e69e0 — Live proxy Challenge bank (task #59).** A Challenge bank (submodules/challenges) exercising the real data plane: HTTP-forward proxy, SOCKS5 proxy, and cache-HIT canaries, scoring PASS only on positive captured evidence (§11.4.69).
- **bf38c35 — HelixQA proxy test bank + runner (task #59).** A HelixQA (submodules/helix_qa) test bank + runner for the proxy, with an honest §11.4.3 SKIP-with-reason when the harness build is unavailable — never a PASS-by-default.

Rev-2 additions — Let's Encrypt Phase 2/3 (task #59)

- **b2afa7d — LE Phase 2+3: hermetic DNS-01 issuance PROVEN end-to-end (task #59).** Real, re-runnable Let's Encrypt DNS-01 certificate issuance against a local Pebble ACME server, fully offline, gated on cert-analyzer verdicts (§11.4.98 / §11.4.107 / §11.4.108). No bluff: Pebble runs PEBBLE_VA_ALWAYS_VALID=0 (genuine challenge validation) and the PASS is scored over the leaf Caddy actually serves. Infra under deploy/letsencrypt/: build.sh (rootless xcaddy build of caddy-challtestsrv:2.8.4 with a post-build list-modules | grep proving the local DNS module linked — build ≠ linked, §11.4.6), Dockerfile.caddy (fixes the v2.8.4 version bug), a libdns DNS-01 provider.go (POSTs set/clear-txt to challtestsrv; no creds; injection-safe; hermetic-test-only), compose.hermetic.yml (Pebble + challtestsrv + CoreDNS + Caddy; Caddy loopback-bound per security-audit M1), a CoreDNS SOA-front that answers the hermetic.test SOA in the ANSWER section so certmagic's DNS-01 zone-determination SOA-walk succeeds (challtestsrv NOTIMPs SOA), and phase3_hermetic_issue.sh (clean-slate re-runnable proof that also avoids certmagic bug #354 via boot ordering). Proof: two clean runs with distinct per-run Pebble intermediate CAs, cert_not_expired / cert_san_matches proxy.hermetic.test / negative-SAN reject / cert_chain_roots_in all PASS; evidence qa-results/letsencrypt/phase3_issuance/{20260701T100119Z,... T100434Z}/. Status docs → Phases 0-3 PASS.
- **022b78d — LE Phase-3 hermetic issuance Challenge, live PASS (task #59).** A HelixQA-style Challenge (challenges/scripts/le_phase3_issuance_challenge.sh, §11.4.4(b) layer 4) driving phase3_hermetic_issue.sh and scoring PASS only on an anti-bluff re-read of the cert-analyzer verdict file (cert_chain_roots_in:PASS AND cert_san_matches:PASS, §11.4.116); honest topology_unsupported SKIP (§11.4.3) when the built image / podman-compose is absent — never a fake pass. Ran LIVE by the conductor: OVERALL=PASS, real cert

(Pebble CA 42f825) issued + verified, run 20260701T101316Z. Also lands the run_proxy_challenges.md §11.4.18 companion missed in f2e69e0.

Rev-2 additions — observability metrics lane (task #56)

- **9b209e9 — Control-plane metrics overlay + RED_MODE scrape guard (task #56).** docker-compose.observability.yml runs the control-plane API as proxy-api with `CONTROL_API_METRICS_ADDR=0.0.0.0:59090` and the network alias matching the committed prometheus.yml target; secrets are NAME-only (§11.4.10) and the base docker-compose.yml stays pristine (§11.4.122). tests/observability/metrics_scrape_test.sh is a §11.4.115 RED_MODE polarity guard: `RED_MODE=1` reproduces "metrics not exposed"; `RED_MODE=0` scrapes /metrics and asserts REAL Prometheus exposition (# HELP/# TYPE + a known metric), asserting content not HTTP 200. Honest boundary (§11.4.6): the API counters are NOT yet byte-path-wired, so the counter-increment-on-traffic sub-proof is an honest feature_disabled_by_config SKIP until that wiring lands (flip via `HELIX_METRICS_BYTEPATH_WIRED=1`). Metrics: `helix_proxy_vpn_up / _acl_decisions_total / _tunnel_down_responses_total`.
- **401234e — Isolated metrics trio: LIVE /metrics scrape PROVEN (task #56).** deploy/observability/compose.metrics.yml — a fully-isolated obs-postgres + obs-redis + obs-api trio (own network, high ports 55432/56379/59091, fresh volume) with a symmetric throwaway test pg password so the control-plane API connects with no dependency on the base-stack password — the #56 live-scrape blocker removed. Ran LIVE by the conductor (the #56 core deliverable): rebuilt the stale control-plane image, booted the trio, obs-api came up serving mTLS + the plaintext /metrics listener on 59091, and metrics_scrape_test.sh PASSED against it — "real Prometheus exposition content: # HELP/# TYPE + helix_proxy_acl_decisions_total + helix_proxy_tunnel_down_responses_total + helix_proxy_vpn_up present". Honest SKIP on the counter-increment sub-proof (byte-path→API increment is P5/P10-pending, metrics.go:14-17, feature_disabled_by_config), never a fake pass. Evidence: qa-results/observability/{scrape,metrics_scrape}/.

fix

All six fixes in this window are **test-suite / anti-bluff-library integrity** fixes (they make the test corpus HONEST — no product/proxy behaviour changes), each rooted per §11.4.102 and closed with §11.4.115 RED→GREEN + §1.1 paired mutation evidence (full detail in docs/issues/fixed/BUGFIXES.md).

- **2e834a7 — BUGFIX-0013:** the CONST-033 no-suspend scanner false-FAILED on the .html/.pdf §11.4.65 export siblings of the governance carriers (CONSTITUTION.md, AGENTS.md, CLAUDE.md, QWEN.md, GEMINI.md), which legitimately quote the banned host-power literals. Made the five governance EXCLUDE_PATHS entries extension-agnostic prefixes; a non-governance file with a banned literal or any real script invocation still trips the scanner (gate not neutered, §11.4.120). Latent — the sibling-blindness class BUGFIX-0011 closed for the bug ledger, one layer over.
- **f219b4d — BUGFIX-0014:** verify-proxy.sh / final-verify.sh classified every through-proxy check as code == expected ? PASS : FAIL, so a momentary outage of the probed *site* false-FAILED a healthy proxy (non-deterministic, not re-runnable). Added a single client-side classifier proxy_conn_verdict in tests/lib/evidence.sh: proxy-reachable → PASS; proxy-miss-but-direct-200 → FAIL (real defect, §11.4.68 not fail-open); both-fail → SKIP (external outage).
- **8219669 — BUGFIX-0015:** ddos_flood_suite scored "survived the flood" PASS with no proof a flood occurred. Added a pure flood_survival_verdict classifier requiring positive captured flood evidence (flood_total > 0 AND flood_responses > 0) before any survival PASS; zero-flood on a listening proxy → FAIL, absent proxy → honest SKIP.
- **80bd833 — BUGFIX-0016:** the benchmark regression ratchet compared against a baseline under the gitignored qa-results/tree, so the comparison never persisted and every run PASSED on the absolute budget regardless of a real regression. Moved the baseline to the committed path tests/dynamic/baselines/benchmark_p95.baseline; seed-once + SKIP on absent, FAIL on p95 growth beyond tolerance, no auto-refresh.
- **9ffbc0a — BUGFIX-0017:** seven proxy canaries in comprehensive-test.sh still used the pre-BUGFIX-0014 blind code != expected ? FAIL pattern. Re-wired each through a conn_check wrapper onto the already-committed proxy_conn_verdict classifier (BUGFIX-0014), so an external outage SKIPS while a real proxy defect still FAILs. The DoH content assertion is preserved via synthetic ANSWER/MISS tokens.
- **39c4139 — BUGFIX-0018:** assert_egress_ip fail-opened the hardest-to-fake VPN-routing §15 proof — when the host's real IP was undeterminable, the egress != host half silently collapsed, so a genuine NO-VPN (egress == host) case could fake-PASS. Now returns exit-2 OPERATOR-BLOCKED when

host_real is not IP-shaped (F-1 hardened it from a two-sentinel deny-list to a positive _evidence_ip_shaped validator). **Also closed the F5/F6 registration gap** — see the guard audit below.

docs

- **fddd25c — Let's Encrypt design + phased plan** for HTTPS with auto-renewal and rotation (informs the Phase-1 analyzer that landed in 2812f48).
- **ae16f61 — CONTINUATION §12.10 resume** refreshed to live state: anti-bluff sweep complete, LE Phase 1 landed, Phases 2/3 in flight.
- **86c1d8b — Release runbook + prerequisite audit** for helix_proxy-0.1.0-dev-0.0.2 (docs/releases/RELEASE_RUNBOOK.md), with the read-only GitHub/GitLab readiness audit and OPERATOR-BLOCKED items OB-1..OB-7.

Rev-2 additions — LE deep-research, security audit + feature ledger

- **6349c0e — LE hermetic + certmagic-#354 + renewal deep-research (§11.4.150).** Three cited deep-research analyses backing the LE workstream: research/certmagic_chain_panic_20260701/ (source-proven root cause of the certmagic v0.21.3 selectPreferredChain empty-chain panic = upstream bug #354, fixed in certmagic v0.25.1 / Caddy 2.11.0; mitigated here via Phase-3 boot ordering), research/letsencrypt_hermetic_20260701/ (the CoreDNS SOA-front design-gap addendum), and research/letsencrypt_renewal_20260701/ (the Phase-5 renewal/rotation procedure, 24 sources).
- **4820da7 — Feature-status ledger + this changelog (Rev 1) + LE security audit (task #59).** Landed docs/features/Status.md + Status_Summary.md (§11.4.153/.56 — 9 PASS-with-evidence / 10 PENDING / 3 OPERATOR-BLOCKED at that point + a §11.4.169 test-type coverage matrix), the Rev-1 draft of THIS changelog, docs/audit/le_hermetic_security_audit_20260701 (0 secrets leaked, CRITICAL 0 / HIGH 0 / MED 1 [Caddy 0.0.0.0 bind — fixed this session] / LOW 3), and docs/helixqa/README.md.
- **7c6b69f — Phase-5 renewal-mechanism correction (§11.4.138).** The Rev-1 renewal doc claimed renewal_window_ratio is a Caddyfile tls subdirective; the conductor's LIVE test disproved it on Caddy 2.8.4 ("unknown subdirective"). Source root cause: that Caddyfile token arrived via PR #7473 (milestone v2.11.1), ~2.5y after 2.8.4. Corrected: in 2.8.4 the ratio is a JSON field apps.tls.automation.policies[].renewal_window_ratio, so the Phase-5 force step is an admin-API PATCH/POST /load — the

Caddyfile line correctly stays commented. Renewal-research doc Rev 2.

- **2f1e49d — Phase-5 live findings: ARI blocks renewal on Pebble 2.6.0.** Conductor live-test (Caddy 2.8.4 + Pebble 2.6.0) disproved three prior assumptions: (1) `renewal_window_ratio=1` via `/load` does NOT force renewal — `certmagic v0.21.3` follows the ARI `selected_time`; (2) a short-lifetime Pebble cert is not renewed either (Pebble 2.6.0 ARI returns a ~2-day window ignoring the short lifetime); (3) the root blocker is ARI, and Pebble 2.6.0 has no `/set-renewal-info` (added v2.8.0). The zero-downtime of the `/load` reload itself IS confirmed (100/100 probes 200). Phase 5 stays honestly PENDING — no renewal proven, never a metadata-only PASS. Renewal-research doc Rev 3.
- **713d4dd — Pebble /set-renewal-info API + Phase-5 certmagic ARI-cache blocker.** docs/research/pebble_set_renewal_info_20260701/ — source-verified (Pebble v2.8.0/v2.10.1) and conductor-LIVE-verified on Pebble 2.10.1: the POST returns 200 and Pebble serves the posted past window at the ARI GET path (the Pebble layer works). BUT `certmagic v0.21.3` (Caddy 2.8.4) caches the ARI persistently and neither POST `/load` nor a Caddy restart forces a re-fetch, so the override never reaches `certmagic` → no renewal. Definitive Phase-5 conclusion: a fast deterministic zero-downtime hermetic renewal needs **Caddy ≥2.11.0 / certmagic ≥v0.25.1** (also fixes #354), which requires adapting the custom DNS provider to `libdns v1.0.0` (Caddy ≥2.10). Also `deploy/letsencrypt/compose.phase5.yml`.
- **2ed8f38 — LE hermetic DNS-01 issuance = PASS in the feature ledger (task #59).** docs/features/Status.md + Status_Summary.md Rev 2 — LE hermetic DNS-01 issuance PENDING → PASS (cites the Phase-3 issuance verdicts, the live Challenge, the RED+GREEN-proven guard, the DNS-module-linked image build); LE renewal/rotation split out as honest PENDING (`certmagic` ARI-cache blocker, needs Caddy ≥2.11.0). Ledger tally: 10 PASS-with-evidence / 9 PENDING / 3 OPERATOR-BLOCKED.

test

Rev-1 note (retained): none of the commits 1-14 carried a bare `test(` Conventional-Commits prefix; the `fix(tests):` ... commits above are classified as fixes (they repair defective tests). Every fix additionally lands or wires its §11.4.135 standing guard — audited below.

Rev-2 additions — Phase-3 guard + observability mTLS generator

- **d2042ac — Standing §11.4.135 regression guard for LE Phase-3 issuance (task #59).** tests/letsencrypt/phase3_issuance_guard.sh — a §11.4.115 RED_MODE polarity guard wrapping phase3_hermetic_issue.sh, registered in run-tests.sh test_regression_guards() with 3-way exit handling (0=PASS, 2=honest topology SKIP when the built image is absent §11.4.3 — collapsing SKIP into FAIL would be a §11.4.1 false-FAIL). Both polarities PROVEN live by the conductor: RED_MODE=0 (GREEN standing guard) → real cert issued + analyzer-verified (run 20260701T101408Z); RED_MODE=1 (reproduce) → broken resolver bypasses the CoreDNS SOA-front → certmagic SOA-walk NOTIMP → Phase-3 FAILS (exit 1) → guard PASS (catches the exact regression the fix closes). Evidence: qa-results/regression/phase3_issuance_guard/.
 - **5d80c00 — Hermetic mTLS test-cert generator for the #56 live-scraper.** tests/observability/gen_test_mtls.sh — generates a self-signed test CA + server cert (CN helix-control-plane, SANs proxy-control-plane/localhost/127.0.0.1/::1, EKU serverAuth) + client cert (CN admin@helix, clientAuth) + a random test pg password into gitignored tests/observability/.mtls/ (§11.4.10 — key material never tracked; .gitignore + umask 077), and prints the four NAME-only podman secret create commands the observability overlay expects. Unblocks the #56 conductor live-scraper (the API is fail-closed: buildTLSConfig runs before the /metrics listener). Hermetically dry-run-verified (openssl verify OK, keypair matches).
-

§11.4.135 regression-guard audit

Scope: every **bugfix** on the release surface (previous tag → HEAD) — BUGFIX-0013 through BUGFIX-0018. For each: the standing guard registered in tests/run-tests.sh → test_regression_guards(), whether that guard is wired with BOTH a GREEN assertion AND a RED_MODE=1 reproduction self-check, and any honest gap.

Evidence basis (§11.4.6 honest boundary). This audit is a documentation pass; the guards were **not re-executed here** (read-only task; the tests/ tree is not run as part of authoring — the §11.4.40 full-suite retest is the operator-run authoritative re-run before the tag, per the runbook). The GREEN + RED status below is established from two captured sources: (a) direct read of tests/run-tests.sh test_regression_guards() (lines 473-699) confirming each guard is wired with both a GREEN invocation and a RED_MODE=1 self-check; (b) the captured RED→GREEN + §1.1 mutation-with-md5-restore runs recorded per fix in docs/issues/

fixed/BUGFIXES.md, including the standing-suite capture at
 BUGFIX-0018 time: tests/run-tests.sh -> 59 run / 53 pass / 6
 skip / 0 fail; BUGFIX-0018 GREEN+RED both PASS.

Fix	Registered guard (tests/ regression/...)	Wired GREEN	Wired RED_MODE=1	Ver
BUGFIX-0013	no_suspend_export_sibling_test.sh (shared; GREEN branch extended for governance-doc siblings)	yes (L572)	yes (L582)	GUA (sha guar
BUGFIX-0014	proxy_conn_verdict_test.sh	yes (L615)	yes (L625)	GUA (ded
BUGFIX-0015	ddos_flood_evidence_test.sh	yes (L635)	yes (L641)	GUA (ded
BUGFIX-0016	benchmark_baseline_ratchet_test.sh	yes (L651)	yes (L657)	GUA (ded
BUGFIX-0017	<i>(no dedicated guard)</i> — reuses proxy_conn_verdict_test.sh (BUGFIX-0014)	via 0014	via 0014	GUA BY-I — se
BUGFIX-0018	assert_egress_ip_host_unknown_test.sh	yes (L669)	yes (L675)	GUA (ded

Additionally, two **Let's Encrypt** features (not bugfixes but shipped
 this window) register standing guards in test_regression_guards():
 the Phase-1 cert_analyzer_selfvalidation_test.sh self-validation
 guard (GREEN L687 + RED_MODE=1 L693, 2812f48), and the Phase-3
 tests/letsencrypt/phase3_issuance_guard.sh DNS-01-issuance guard
 (d2042ac) — a §11.4.115 RED_MODE polarity guard with 3-way exit
 handling (0=PASS, 2=honest topology SKIP §11.4.3), both
 polarities proven live by the conductor (GREEN run
 20260701T101408Z; RED reproduces the SOA-front-bypass
 regression).

All 12 guard files referenced by test_regression_guards() are
 present on disk under tests/regression/ (verified). Each is wired
 GREEN + RED (the RED self-check enforces §11.4.7 — a guard
 that cannot reproduce its defect is itself a finding).

Tally

- Release-window bugfixes audited: **6** (BUGFIX-0013...0018).
- With a registered standing guard (dedicated or shared), wired
 GREEN + RED: **5** (0013 shared, 0014, 0015, 0016, 0018).
- Guarded-by-reuse (no dedicated guard; documented; runtime
 proof captured): **1** (0017).
- **True unguarded GAPS: 0.**

Honest findings (§11.4.6 — not papered over)

1. **BUGFIX-0017 has no dedicated standing guard (documented reuse, not a silent gap).** Per its own ledger entry, BUGFIX-0017 only re-wires comprehensive-test.sh's canaries onto the *already-guarded* classifier proxy_conn_verdict (covered by BUGFIX-0014's registered proxy_conn_verdict_test.sh, full truth-table + RED/GREEN polarity + §1.1 mutation). Its own lane-specific runtime proof is the captured conductor live smoke at qa-results/regression/comprehensive_f1_conductor_smoke/ (CASE1 working-proxy → PASS, CASE2 simulated outage → SKIP:network_unreachable_external). This is a legitimate, documented reuse — the invariant IS guarded — but it is surfaced here honestly rather than counted as a dedicated guard.
 2. **A real §11.4.135 registration gap existed *within* this window and was CLOSED within it.** The F5 (ddos_flood_evidence_test.sh, BUGFIX-0015) and F6 (benchmark_baseline_ratchet_test.sh, BUGFIX-0016) guards were committed to disk with their fixes but were **not wired** into test_regression_guards(), so for two commits (8219669, 80bd833) they never ran on a build — a live gap. BUGFIX-0018 (39c4139) surfaced and closed it, registering F5 + F6 + F7 (each GREEN + RED_MODE=1). As of HEAD both are wired (L635/L641 and L651/L657). Net state at the release candidate: no on-disk-but-unwired guard remains among the audited set. This is recorded so the history is not smoothed over.
 3. **Re-execution deferred to the release gate.** No guard was re-run in this authoring pass (see Evidence basis). The §11.4.40 full-suite retest on a clean baseline — the operator-run release gate documented in the runbook (Step 2) — is the authoritative re-run that must be GREEN before the helix_proxy-0.1.0-dev-0.0.2 tag is created. This changelog does not assert a fresh run it did not perform.
-

Known-pending at this tag (§11.4.6 honest boundary)

What genuinely shipped-and-PROVEN this window vs what is diagnosed-but-not-yet-done. None of the below is a bluff PASS — each is an honestly-tracked gap.

- **LE Phase 5 (renewal / rotation) — diagnosed but PENDING.** Hermetic DNS-01 *issuance* (Phases 0–3) is live-PROVEN; automatic *renewal/rotation* is NOT. The conductor's live Phase-5 tests (2f1e49d, 713d4dd) root-caused the blocker to

certmagic's persistent **ARI cache**: on Caddy 2.8.4 / certmagic v0.21.3 neither `renewal_window_ratio=1` via `POST /load`, a Caddy restart, nor a Pebble `/set-renewal-info` override (verified working at the Pebble layer on 2.10.1) forces certmagic to re-fetch the renewal window, so no renewal fires. Definitive path: needs **Caddy ≥2.11.0 / certmagic ≥v0.25.1** (which also fixes bug #354), which in turn requires adapting the custom `challtestsrv` DNS provider to `libdns v1.0.0` (Caddy ≥2.10). Phase 5 stays honestly **PENDING** — never a metadata-only **PASS**.

- **LE Phase 4 and Phase 6 — OPERATOR-BLOCKED.** Per the LE phased plan, Phase 4 (real public Let's Encrypt issuance against a real domain) and Phase 6 (the production HTTPS proxy front-end cutover) require operator-owned inputs (a public domain + DNS control, production topology decisions) and remain **OPERATOR-BLOCKED** (§11.4.21) — outside the autonomous hermetic surface.
 - **Observability #56 — counter-increment-on-traffic sub-proof PENDING byte-path wiring.** The `LIVE /metrics` scrape is **PROVEN** (401234e — real Prometheus exposition served + asserted). What remains is the `byte-path` → API counter increment: the API metric counters are not yet wired into the proxy data path, so the "counter rises when real traffic flows" sub-proof is an honest `feature_disabled_by_config SKIP` (`metrics.go:14-17`), flippable to a real assertion via `HELIX_METRICS_BYTEPATH_WIRED=1` once that wiring lands (P5/P10).
-

Sources

- `docs/issues/fixed/BUGFIXES.md` (BUGFIX-0013...0018 entries — root cause, RED→GREEN, §1.1 mutation md5 restores, captured evidence paths).
- `tests/run-tests.sh` → `test_regression_guards()` (lines 473-699 — guard wiring).
- `docs/releases/RELEASE_RUNBOOK.md` (prefix resolution, tag naming, §11.4.40 gate, **OPERATOR-BLOCKED** items).
- `git log helix_proxy-0.1.0-dev-0.0.1..HEAD` (commit inventory).